

BECKMAN COULTER LIFE SCIENCES

Biomek JSON Add-on Steps

Step Types Contributed by Add-on Modules and Device
Integrations

JSON format version 1.0

2 documents

Companion volumes: Biomek JSON Authoring Guide (in *Biomek JSON Authoring Guide*) · Biomek Instrument
& Project Reference (in *Biomek Instrument & Project Reference*)

Table of Contents

Add-on Steps

Add-on Steps

Guided Setup

Add-on Steps

Biomek Software includes a core set of built-in step types, documented in the step documentation index (in *Biomek JSON Authoring Guide*). Beyond that core set, **add-on modules, device integrations, and site plug-ins register their own step types**. This area holds the reference documentation for those steps.

An add-on step behaves like any other step in the method tree — same envelope, same base keys, same structural rules — with two differences worth knowing before you author one:

- **Its `stepType` may be a friendly name or a COM ProgID, depending on the add-on.** Many add-ons publish a short, friendly `stepType`; one that does not carries its registration identifier (a ProgID) instead. Import resolves a `stepType` against the built-in registry first, then as a CLSID string, then as a ProgID — so a ProgID is a legal `stepType`. Check the add-on's own documentation for the exact `stepType` it expects. See *Beyond the Built-in Steps* in Method JSON Structure (in *Biomek JSON Authoring Guide*).
- **The add-on must be installed on the target instrument.** An add-on step imports only on an install where its library is registered. On an instrument without it, import fails with an unresolved-CLSID error. Confirm the target install before authoring one.

Shared conventions (universal base keys, expression syntax, labware/position resolution, probe/mandrel selection, the variable environment) are the same as for built-in steps and live in Base Keys & Serialization Conventions (in *Biomek JSON Authoring Guide*) and in `../concept-guides/` (in *Biomek JSON Authoring Guide*).

Documented add-on steps

Step	stepType	Requires	What it does
Guided Setup	"Guided Labware Setup"	Guided Labware Setup add-on	Configures the deck at method start with a guided labware placement dialog; loads a named layout, verifies pod configuration, and pauses for operator confirmation.

Add-on steps not documented here

If you need to author an add-on step that this area does not cover, obtain its parameter reference from Beckman Coulter — or, for a third-party module, from the add-on's vendor — rather than inferring it from an exported method. An add-on step can carry structural or parameter requirements that are not evident from a sample export.

[↑ Table of contents](#)

Guided Setup

Property	Value
stepType	"Guided Labware Setup"
Category	Leaf (SubStepsReadOnly)
Terminator	N/A
Compatible Hardware	All (requires the Guided Labware Setup add-on installed and registered)

Base keys + casing: see Base Keys & Serialization Conventions (in *Biomek JSON Authoring Guide*).

Add-on prerequisite. Guided Setup is an add-on step rather than a built-in, and lives in `Biomek5GuidedLabwareLib.dll`. The target Biomek install **must have this library installed and registered** for import to succeed — on a stock Biomek without the library, authoring this step fails at import as an unresolved-CLSID error. If the target lacks the library, use plain Instrument Setup (see Instrument Setup (in *Biomek JSON Authoring Guide*)) instead.

The minimal authored form:

```
{
  "stepType": "Guided Labware Setup",
  "parameters": {
    "deck": "i7 Standard"
  }
}
```

Behavior Summary

The Guided Setup step configures the instrument deck at the start of a method run. It verifies pod configuration, loads a named deck layout — preserving labware at any "AsIs" positions and clearing the remaining ones — and runs custom setup scripts. At the end it presents the operator with a guided labware placement dialog, pausing the run for confirmation.

It is an ordinary registered step — any Biomek install that has the add-on registered can run it, in any method. In the editor it appears on the "Setup & Device Steps" palette. Along the way it also handles labware group management, barcode scanning, deck light control, pod parking, and light curtain/interlock coordination.

Parameters Reference Table

Core Parameters

Key	Type	Units	Required	Default	Description
<code>deck</code>	<code>string</code>	—	Yes	—	Deck layout name. Must name a deck layout defined on the instrument.
<code>groups</code>	<code>array</code>	—	No	—	Array of labware group configuration objects. Each group has <code>name</code> , <code>labwareList</code> , and <code>notes</code> .
<code>asIsPositions</code>	<code>array</code>	—	No	<code>[]</code> (<i>empty list</i>)	List of deck position names whose existing labware should be preserved during layout loading.
<code>initialKey</code>	<code>string</code>	—	No	<code>""</code>	Setup notes text shown to the operator before confirmation. Supports expression evaluation.
<code>permanentLabware</code>	<code>object</code> (<code>dictionary</code>)	—	No	<code>{}</code> (<i>empty dictionary</i>)	Keyed by deck position name and contains a subdictionary with only the <code>LiquidType</code> key and value associated with the permanent labware at that position. Populated at design time from deck positions that carry the

Key	Type	Units	Required	Default	Description
					instrument's own permanent-labware flag; the LiquidType may be modified, but no other parameters should be added to the subdictionary for each position. The permanent labware associated with the deck position will be used with the correct liquid type as provided upon import.

Script Parameters

Key	Type	Units	Required	Default	Description
<code>code</code>	<code>string</code>	—	No	<code>""</code>	Custom script code to execute during setup (before labware placement).
<code>isVBScript</code>	<code>boolean</code>	—	No	<code>false</code>	If <code>true</code> , <code>code</code> is VBScript; if <code>false</code> , JavaScript.

Confirmation & Behavior Parameters

Key	Type	Units	Required	Default	Description
<code>confirmSetup?</code>	<code>boolean</code>	—	No	<code>true</code>	If <code>true</code> , pause for operator to confirm deck setup.
<code>verifyPodSetup?</code>	<code>boolean</code>	—	No	<code>true</code>	If <code>true</code> , verify pod configuration before proceeding.
<code>turnDeckLightOn?</code>	<code>boolean</code>	—	No	<code>true</code>	If <code>true</code> , turn on deck light during setup.

Pod Setup Parameters (nested inside `podSetup` dictionary)

Key	Type	Units	Required	Default	Description
podSetup	object	—	No	—	Dictionary containing pod configuration to verify.
podSetup.leftHasTips	boolean	—	No	—	Left pod should have tips loaded.
podSetup.rightHasTips	boolean	—	No	—	Right pod should have tips loaded.
podSetup.leftTipType	string	—	No	—	Expected left tip type.
podSetup.rightTipType	string	—	No	—	Expected right tip type.
podSetup.washLiquidType	string	—	No	—	Recognized key name, but not read by pod verification . Do not author it; preserve it verbatim if an existing method already carries it.

Scope of podSetup verification. Runtime pod verification reads exactly four keys: `leftHasTips`, `rightHasTips`, `leftTipType`, `rightTipType`. For each pod it brings the modeled tip state in line with those keys and appends a sentence describing the result to the setup dialog text the operator sees. Independently of any key, it also releases anything the pod's gripper is modeled as holding and, if it was holding something, adds "Ensure the ... pod is not holding any labware." to that same text.

Gripper- and tool-verification key names — `checkLeftGripper`, `checkRightGripper`, `leftHasTool`, `leftToolType`, `rightHasTool`, `rightToolType`, `toolSourceLoc`, `toolSourceSlot` — exist as recognized names but are **not** part of runtime pod verification. **Do not author them.**

UI State Parameters (design-time only)

Key	Type	Units	Required	Default	Description
customColors	variant list	—	No	(empty list)	UI color configuration. Editor-written; not read at runtime.

Key	Type	Units	Required	Default	Description
					Written on every editor save — see the note below the table.

Universal base keys (`caption` , `dynamic?` , `stepUI` , `isPreconfigured` , ...) apply to this step but are omitted here — see *Base Keys & Serialization Conventions (in Biomek JSON Authoring Guide)*. This step can carry `caption` and `isPreconfigured` ; both are editor state — preserve them verbatim if present, never invent them.

Note — why these keys are Optional but always present in editor-produced steps.

Every parameter documented above is written **unconditionally** by the step's editor panel each time the step is saved: `deck` , `groups` , `asIsPositions` , `initialKey` , `permanentLabware` , `code` , `isVBScript` , `confirmSetup?` , `verifyPodSetup?` , `turnDeckLightOn?` , `podSetup` , `customColors` . So a step authored in the editor carries the whole set.

That is a property of the editor, not a requirement of the run. Enqueue reads every one of them through a defaulting accessor or an `IsBound` guard, so the step still enqueues with any of them absent — `turnDeckLightOn?` , for instance, falls back to `true` . Keep them all Optional, and preserve exactly what an existing method carries when round-tripping — do not add a key a method does not already have.

Labware Group Structure

Each element in the `groups` array is a dictionary with:

Key	Type	Description
<code>name</code>	<code>string</code>	Group name (must be unique across all groups)
<code>labwareList</code>	<code>array</code>	Labware items in this group. Emit <code>[]</code> when authoring a new group — see below.
<code>notes</code>	<code>string</code>	Operator notes for this group

Labware items — do not hand-author

Each element of `labwareList` is a **fully serialized labware object**, not a small descriptor. In real methods an item carries the labware class reference, position, volume type, per-well

amount and liquid-type arrays (96+ entries each), a `properties` sub-dictionary (name, barcode, liquid type, device, sense flag), and a set of internal markers that Guided Setup writes itself:

Marker	Role
<code>_is_brt_lwindex</code>	Internal identity of the item within its group
<code>position</code>	Target deck position (may be an expression)
<code>_is_brt_stackdepth</code>	Stack depth at that position; <code>1</code> is the bottom item
<code>use</code>	Whether the item is used on this run (may be an expression)
<code>wellNotesLine1</code>	Per-well label text shown in the placement dialog
<code>wellColors</code>	Per-well RGB values sent to the display
<code>notes</code>	Operator notes for this item
<code>hasBarcode</code>	Pause for a barcode read on this item
<code>advancedSetup</code>	Item still requires operator setup
<code>_is_brt_valid_pos</code>	Written by the step: whether the resolved position exists on the deck
<code>wellNotesLine2</code>	Per-well "line 2" label text shown in the placement dialog
<code>_is_brt_colorsource</code>	Recognized marker name with no reader or writer in the current step; preserve verbatim if present

Do not author `labwareList` entries or any of the above markers by hand — populated groups are produced by the Guided Setup editor, and a partially-formed item will not round-trip. Emit `"labwareList": []` for a new group, and when round-tripping an existing method, preserve every labware item and every marker verbatim.

Cross-Field Validation Rules

- `deck` **must not** be empty (fails as a no-deck-selected error).
- `deck` **must** name a deck layout defined on the instrument (fails as a deck-not-defined error).
- All group `name` values **must be unique** (fails as a duplicate-group-name error).
- If `confirmSetup?` is `false` and any labware requires barcode input, the step fails as a barcode-requires-confirmation error. Note that this includes labware with the TubeScanRack characteristic even if `hasBarcode` is false. This is **not** a design-time

guard: it is raised late in the enqueue sequence, after the deck has been loaded, groups processed, and setup scripts run — so earlier side effects are already queued when the error surfaces. Author `confirmSetup?: true` whenever any labware carries barcode input.

Enqueue Sequence

1. **Setup notes** — Evaluates `initialKey` as an expression and seeds the operator message with the result.
2. **Pod speed** — Sets Pod1 and Pod2 speeds to 100 (if present).
3. **Pod setup verification** — If `verifyPodSetup?` is `true` and `podSetup` is bound, brings the modeled pod state in line with the four tip keys and appends a description to the operator message.
4. **Deck validation and deck-name warning** — Rejects an empty or undefined `deck` ; then, if the currently-loaded deck has a different name, appends a warning to the operator message. This happens **before** any deck loading, group processing, or script execution.
5. **Deck loading** — Loads the named deck layout and removes labware from positions that are neither in `asIsPositions` nor flagged permanent on the deck position itself.
6. **Labware group processing** — Creates labware groups from `groups` , validates unique names, fills labware dictionaries.
7. **Script execution** — Runs custom `code` script, then any plugin scripts installed at `%ProgramData%\Beckman Coulter\Guided Labware Setup\Scripts\*.vbs` .
8. **Barcode/confirmation check** — If `confirmSetup?` is `false` and any labware requires barcode input, raises the error described under Cross-Field Validation.
9. **Deck light** — Sets the deck light to `turnDeckLightOn?` . Skipped entirely when simulating.
10. **Pod parking** — Parks Pod1 and Pod2 at their park locations. **Skipped when simulating or when `confirmSetup?` is `false` .**
11. **Guided action** — Enqueues the interactive labware-placement dialog (pauses the light curtain/interlock, presents the UI, resumes). **Skipped when simulating or when `confirmSetup?` is `false` .**

In simulation the step still evaluates notes, loads the deck, processes groups, and runs scripts — it just does not touch the deck light, does not park pods, and does not enqueue the operator dialog.

Structural Context

This step is a leaf (SubStepsReadOnly — no user-editable sub-steps).

Canonical Examples

Basic guided setup:

```
{
  "stepType": "Guided Labware Setup",
  "parameters": {
    "deck": "i7 Standard",
    "confirmSetup?": true,
    "verifyPodSetup?": true,
    "turnDeckLightOn?": true
  }
}
```

Guided setup with groups and AsIs positions:

```
{
  "stepType": "Guided Labware Setup",
  "parameters": {
    "deck": "i5 Span-8",
    "asIsPositions": ["P1", "P2"],
    "groups": [
      {
        "name": "Source Plates",
        "labwareList": [],
        "notes": "Place source plates in positions P3-P5"
      },
      {
        "name": "Destination Plates",
        "labwareList": [],
        "notes": "Place destination plates in positions P6-P8"
      }
    ],
    "initialKey": "Ensure tip boxes are full before starting.",
    "confirmSetup?": true,
    "verifyPodSetup?": false,
    "turnDeckLightOn?": true
  }
}
```

Common Mistakes

- Disabling confirmation with barcode input: if any labware needs barcode scanning, `confirmSetup?` must be `true`. Enqueue rejects the combination outright, and does

so *after* the deck load and setup scripts have already been queued.

- Fabricating `podSetup.*` gripper/tool keys: pod verification acts on four keys only — `leftHasTips` , `rightHasTips` , `leftTipType` , `rightTipType` . `checkLeftGripper` , `leftToolType` , `toolSourceLoc` and the rest are recognized names but are not part of verification; `washLiquidType` is likewise inert. Do not author any of them.
- **Hand-authoring a populated groups entry:** labware items are full serialized labware objects with internal markers. Emit `"labwareList": []` and let the editor populate it; preserve existing items verbatim.
- **Assuming the guided dialog always runs:** pod parking and the operator dialog are skipped in simulation and whenever `confirmSetup?` is `false` , even though the deck load and scripts still execute.

↑ Table of contents